

Competitive Programming

Section 34 Editorial: Probability and Expected Value

Competitive Programming Course

2026-08-13

Table of contents

1	Editorial overview	3
2	A. Expected Prize	4
2.1	Restatement and constraints	4
2.2	Derivation	4
2.3	Worked trace	4
2.4	Algorithm and proof	4
2.5	Complexity, language notes, and tests	5
2.6	Full C++ solution	5
2.7	Full Python solution	5
3	B. First Success	7
3.1	Restatement and constraints	7
3.2	Derivation and trace	7
3.3	Algorithm and proof	7
3.4	Complexity, language notes, and tests	7
3.5	Full C++ solution	8
3.6	Full Python solution	8
4	C. Coupon Collector	10
4.1	Restatement and constraints	10
4.2	Derivation and complete trace	10
4.3	Algorithm	10
4.4	Correctness proof	11
4.5	Complexity, language notes, and tests	11
4.6	Full C++ solution	11
4.7	Full Python solution	12
5	D. Simultaneous Wins	13
5.1	Restatement	13
5.2	Constraints and traps	13
5.3	Observation and trace	13
5.4	Algorithm	13
5.5	Correctness proof	14
5.6	Complexity	14
5.7	Full C++ solution	14
5.8	Full Python solution	15
5.9	Implementation notes and tests	16
5.10	Stress testing	16
6	Local workflow	17
7	What to carry forward	18

1 Editorial overview

Problem	Random-variable model	Main tool	Cost
A. Expected Prize	one reward variable per event	linearity	$O(n \log \text{MOD})$
B. First Success	expected remaining attempts	first-step equation	$O(t \log \text{MOD})$
C. Coupon Collector	wait for each new type	staged linearity	$O(\max n + q)$
D. Simultaneous Wins	winning-pair indicators	running probability sum	$O(n \log \text{MOD})$

All answers are rational numbers represented modulo the prime $\text{MOD} = 1,000,000,007$. The local denominators are strictly below MOD , so their Fermat inverses exist.

2 A. Expected Prize

2.1 Restatement and constraints

There are up to 200,000 bonus events. Event i awards value v_i with probability p_i/q_i and awards zero otherwise. The statement calls events independent, but the solution only needs their individual probabilities and q_i not divisible by MOD.

2.2 Derivation

Define X_i to be the points from event i . It is v_i with probability p_i/q_i and zero otherwise, hence $E[X_i] = v_i * p_i / q_i$. The total score is the sum of the X_i , so linearity gives

$$E[\text{total}] = \sum v_i * p_i * \text{inverse}(q_i) \text{ modulo MOD}.$$

Linearity does not require independence. This is why we sum event contributions instead of enumerating all 2^n possible subsets.

2.3 Worked trace

For sample events (1,2,10), (1,4,8), and (3,4,12):

$$\begin{aligned} E[X_1] &= 10 * 1/2 = 5 \\ E[X_2] &= 8 * 1/4 = 2 \\ E[X_3] &= 12 * 3/4 = 9 \\ E[\text{total}] &= 16 \end{aligned}$$

If the third event were correlated with the first two but still had probability 3/4, the same expected total would follow. Independence would matter for a product of probabilities, not this sum.

2.4 Algorithm and proof

```
answer = 0
for each p,q,v:
    answer += (v modulo MOD) * (p modulo MOD) * q^(MOD-2)
    answer %= MOD
print answer
```

For each event this expression is its exact expected contribution in the finite field. By linearity, summing all contributions equals the expected total. The inverse is valid because $1 \leq q < \text{MOD}$. Therefore the algorithm is correct. QED.

2.5 Complexity, language notes, and tests

Time is $O(n \log \text{MOD})$, memory $O(1)$ besides input/output buffering. C++ products must use long long. Python should use the three-argument pow function; ordinary division and floating point both lose the exact residue.

Test one certain bonus, a zero-probability bonus, a zero value, and a value larger than MOD. Frequent wrong answers multiply all event probabilities or incorrectly think dependent events invalidate linearity.

2.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    long long ans = 0;
    for (int i = 0; i < n; i++) {
        long long p, q, v;
        cin >> p >> q >> v;
        ans = (ans + (v % MOD) * (p % MOD) % MOD * mod_pow(q, MOD - 2)) % MOD;
    }
    cout << ans << '\n';
    return 0;
}
```

2.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    ans = 0
    for _ in range(n):
        p, q, v = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        ans = (ans + v % MOD * p % MOD * pow(q, MOD - 2, MOD)) % MOD
    print(ans)

if __name__ == "__main__":
    main()

```

3 B. First Success

3.1 Restatement and constraints

For each of up to 200,000 queries, independent attempts succeed with probability p/q , where $1 \leq p \leq q < \text{MOD}$. Find the expected attempts through the first success. The lower bound $p \geq 1$ is essential: probability zero would mean an infinite waiting time, not answer zero.

3.2 Derivation and trace

Let E be the expected attempts before an attempt. One attempt is made now. If it fails, probability $(q-p)/q$, the state is exactly the same as before:

$$\begin{aligned} E &= 1 + ((q-p)/q) E \\ (p/q) E &= 1 \\ E &= q/p \end{aligned}$$

For $p=2$, $q=5$:

$$\begin{aligned} E &= 1 + (3/5)E \\ (2/5)E &= 1 \\ E &= 5/2 \end{aligned}$$

The modular output is 5 times $\text{inverse}(2)$, namely 500000006.

3.3 Algorithm and proof

For every query, print $q * p^{-1} \pmod{\text{MOD}}$. The first-step equation conditions on all possible results of the first attempt and uniquely solves to q/p . Since p is nonzero modulo MOD , its inverse represents that rational value exactly. Thus the algorithm is correct. QED.

3.4 Complexity, language notes, and tests

Time is $O(t \log \text{MOD})$, memory $O(1)$ besides output. In C++, the product of two reduced residues fits in signed 64-bit. In Python, $\text{pow}(p, \text{MOD}-2, \text{MOD})$ is the right inverse operation.

Test $p=q$, $p=1$, $p=1$ and $q=2$, and large p and q close to MOD . Wrong submissions usually output p/q , omit the leading one in the recurrence, or treat $p=0$ as a valid zero expectation.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        long long p, q;
        cin >> p >> q;
        cout << q % MOD * mod_pow(p, MOD - 2) % MOD << '\n';
    }
    return 0;
}
```

3.6 Full Python solution

```
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out = []
    for _ in range(t):
        p, q = data[idx], data[idx + 1]
```

```
        idx += 2
        out.append(str(q % MOD * pow(p, MOD - 2, MOD) % MOD))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

4 C. Coupon Collector

4.1 Restatement and constraints

For each of up to 200,000 queries, n is at most one million. Each draw returns one of n coupon types uniformly and independently. We need the expected draws until every type has appeared. Repeating a computation for each query is too slow, but all queries share harmonic prefixes.

4.2 Derivation and complete trace

At stage i , exactly i types are already known. A draw finds a new type with probability $(n-i)/n$, so its geometric waiting-time expectation is $n/(n-i)$. Adding all stages gives

$$\begin{aligned} E &= n/n + n/(n-1) + \dots + n/1 \\ &= n * (1 + 1/2 + \dots + 1/n). \end{aligned}$$

For $n=3$:

stage	success probability	expected draws
0 -> 1	3/3	1
1 -> 2	2/3	3/2
2 -> 3	1/3	3
total		11/2

The required residue is 3 times $(1 + \text{inverse}(2) + \text{inverse}(3))$, equal to 500000009. The stage random variables are added with linearity.

4.3 Algorithm

Read all queries and let limit be their maximum. Precompute inverses and harmonic prefixes in linear time:

```
inv[1] = 1
H[0] = 0
for i from 1 to limit:
    if i >= 2:
        inv[i] = MOD - (MOD/i) * inv[MOD%i] modulo MOD
        H[i] = H[i-1] + inv[i] modulo MOD
for n in queries:
    print n * H[n] modulo MOD
```

The inverse recurrence is valid for $1 \leq i < \text{MOD}$, precisely the local bound. It is preferable to one modular exponentiation per i .

4.4 Correctness proof

During stage i , the probability of moving to stage $i+1$ is $(n-i)/n$, so the First Success result gives an expected stage length $n/(n-i)$. Completion is the sum of exactly these n stages. Linearity gives n times the harmonic sum. The precomputation stores every required modular reciprocal and prefix sum, so each answer is that exact rational expectation modulo MOD. QED.

4.5 Complexity, language notes, and tests

Preprocessing costs $O(\max n)$, each query $O(1)$, and memory $O(\max n)$. C++ should cast the inverse-recurrence product to long long. Python lists are practical at one million entries. A pow-per-inverse loop is correct but $O(\max n \log \text{MOD})$ and does unnecessary work.

Test $n=1$, $n=2$, $n=3$, consecutive queries, and the maximum query. Common wrong answers use n/i instead of $n/(n-i)$, omit the first stage, or recompute a harmonic sum independently per query.

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    int limit = 0;
    for (int &n : values) {
        cin >> n;
        limit = max(limit, n);
    }
    vector<long long> harmonic(limit + 1, 0);
    for (int i = 1; i <= limit; i++) {
```

```

        harmonic[i] = (harmonic[i - 1] + mod_pow(i, MOD - 2)) % MOD;
    }
    for (int n : values) cout << n * harmonic[n] % MOD << '\n';
    return 0;
}

```

4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    values = data[1 : 1 + q]
    limit = max(values)
    inv = [0] * (limit + 1)
    harmonic = [0] * (limit + 1)
    for i in range(1, limit + 1):
        inv[i] = pow(i, MOD - 2, MOD)
        harmonic[i] = (harmonic[i - 1] + inv[i]) % MOD
    print("\n".join(str(n * harmonic[n] % MOD) for n in values))

if __name__ == "__main__":
    main()

```

5 D. Simultaneous Wins

5.1 Restatement

Independent game i is won with probability $p[i] / q[i]$. Count every unordered pair of games that are both won, then print the expectation of that count modulo MOD .

5.2 Constraints and traps

n can be 200000, so checking all $O(n^2)$ pairs is too slow. The result is an exact rational number, not a decimal. Each $q[i]$ is nonzero modulo the prime MOD , so represent each probability with its Fermat inverse.

Independence matters here: for two distinct games, the probability that both win is the product of their probabilities. A game with probability zero simply contributes no winning pair.

5.3 Observation and trace

For each pair $i < j$, define an indicator that is one precisely when both games win. Linearity gives the expected answer as the sum of all pair probabilities:

```
sum over  $i < j$  of  $\text{probability}[i] * \text{probability}[j]$ 
```

Process games left to right. If `sum_previous` is the sum of earlier win probabilities, a new chance p contributes $p * \text{sum_previous}$, then joins the sum.

For probabilities $1/2$, $1/3$, 1 :

```
new 1/2: contribution 0
new 1/3: contribution (1/3)*(1/2) = 1/6
new 1:   contribution 1*(1/2 + 1/3) = 5/6
total = 1
```

5.4 Algorithm

```
sum_previous = answer = 0
for each (p, q):
    chance = p * inverse(q) mod MOD
    answer += chance * sum_previous
    sum_previous += chance
```

```
    reduce both modulo MOD
print answer
```

5.5 Correctness proof

Lemma. Before processing game j , `sum_previous` equals the sum of the win probabilities of exactly games $1..j-1$, and `answer` equals the expected number of winning pairs wholly among those games.

Proof. Both values are zero before game one. When adding game j , each new pair is exactly (i, j) for one earlier game i . Independence gives its probability as `chance * probability[i]`, whose sum is `chance * sum_previous`. Adding that preserves the answer invariant, and then adding `chance` preserves the probability-sum invariant. $\square$

Theorem. The algorithm prints the expected number of unordered pairs of games that are both won.

Proof. After all games, the lemma says `answer` sums the probability of every unordered winning-pair indicator exactly once. By linearity of expectation, that sum is the requested expectation. $\square$

5.6 Complexity

Each modular inverse uses binary exponentiation.

```
Time:    O(n log MOD)
Memory:  O(1) besides input/output buffering
```

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007;

long long mod_pow(long long base, long long exponent) {
    long long result = 1;

    while (exponent > 0) {
        if (exponent & 1)
            result = result * base % MOD;

        base = base * base % MOD;
        exponent >>= 1;
    }

    return result;
}
```

```

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    long long probability_sum = 0;
    long long answer = 0;

    while (n--) {
        long long numerator, denominator;
        cin >> numerator >> denominator;

        long long probability = (
            numerator * mod_pow(denominator, MOD - 2) % MOD
        );

        answer = (
            answer + probability * probability_sum
        ) % MOD;

        probability_sum = (
            probability_sum + probability
        ) % MOD;
    }

    cout << answer << '\n';
}

```

5.8 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    probability_sum = 0
    answer = 0

    for index in range(data[0]):

```

```

p, q = data[1 + 2 * index:3 + 2 * index]
probability = p * pow(q, MOD - 2, MOD) % MOD

answer = (
    answer + probability * probability_sum
) % MOD
probability_sum = (
    probability_sum + probability
) % MOD

print(answer)

if __name__ == "__main__":
    main()

```

5.9 Implementation notes and tests

- Use `long long` for modular products in C++; use `pow(q, MOD-2, MOD)` in Python.
- Do not add `chance * chance`: pairs require two distinct games.
- Test one game, probability zero, all certain wins, and mixed fractions.

5.10 Stress testing

For at most ten games, enumerate every win/loss outcome with exact rational `Fraction` arithmetic, count its winning pairs, and weight it by its outcome probability. This does not reuse the running-sum formula. The scale case has 200000 games of probability $1/2$, whose expected answer is $C(200000, 2) / 4$ modulo MOD.

6 Local workflow

Run:

```
CP_TARGET=solution python3 sections/34_probability_expected_value/check.py
```

For small verification, use exact rational arithmetic. Enumerate a few reward outcomes, solve the first-step equation symbolically, and compare coupon stage sums using Python Fraction before reducing numerator times inverse(denominator) modulo MOD.

7 What to carry forward

Define the random variable or expected remaining state first. Linearity avoids distribution enumeration, and a first-step equation exposes every self-loop.